

Transformations de type *mapper*

Module 4 — map, flatMap, select, withColumn, UDF et Pandas UDF

Objectifs pédagogiques

- Distinguer transformations lazy et actions eager, et comprendre leurs implications
- Appliquer les transformations `map()` et `flatMap()` sur des RDD
- Manipuler les colonnes d'un DataFrame avec `select()`, `withColumn()`, `filter()`
- Créer et utiliser des UDF (User-Defined Functions) simples et vectorisées
- Choisir la bonne transformation selon le contexte et les contraintes de performance

1. Transformations lazy vs actions eager

Transformations lazy vs actions eager

Principe central — aucun calcul n'est effectué avant une action

Transformations (lazy)

Retournent un nouveau RDD ou DataFrame — rien n'est calculé

- `map()` → nouveau RDD/DF
- `filter()` → nouveau RDD/DF
- `flatMap()` → nouveau RDD/DF
- `select()` → nouveau DF
- `withColumn()` → nouveau DF
- `groupBy()` → `RelGroupedDF`
- `join()` → nouveau DF

Actions (eager)

Déclenchent l'exécution complète du DAG

- `count()` → Long
- `collect()` → List
- `show()` → affichage
- `take(n)` → List
- `first()` → Row
- `write()` → fichier
- `toPandas()` → DataFrame Pandas

Avantage : Catalyst peut fusionner filter + select, pousser la projection au niveau Parquet, et appliquer les filtres avant tout chargement en mémoire — sans lazy evaluation, chaque transformation nécessiterait un passage complet sur toutes les données.

Lazy evaluation en pratique

Catalyst optimise l'ensemble du pipeline avant d'exécuter le moindre calcul

```
# Ce pipeline NE CALCULE RIEN à ce stade
df = spark.read.parquet("data/transactions/")      # lecture différée
df = df.filter(F.col("montant") > 100)             # filtrage différé
df = df.select("client_id", "montant")             # projection différée
df = df.withColumn("tva", F.col("montant") * 0.2)   # calcul différé

df.show()     # ← ici seulement Spark lit, filtre, projette ET calcule la TVA en un seul passage
```

Forcer la matérialisation avec cache()

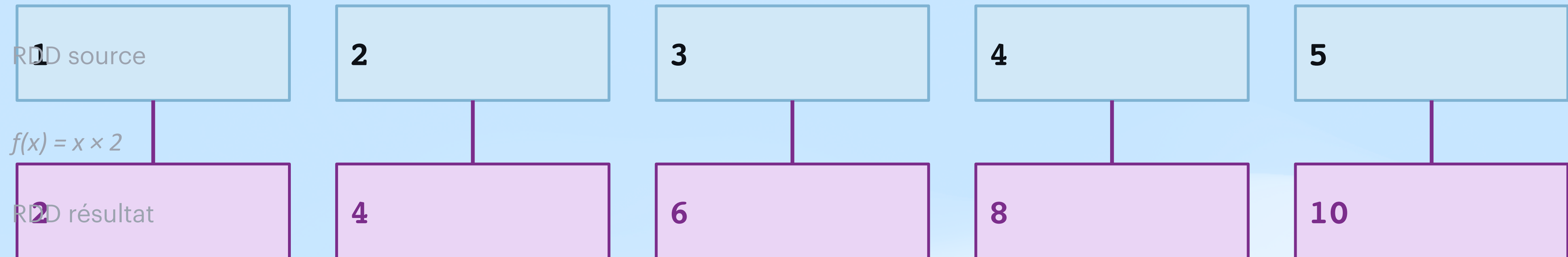
```
df_prepare = df.filter(...).withColumn(...).cache()
df_prepare.count()    # Force la matérialisation et le stockage en mémoire

# Maintenant df_prepare est en mémoire – show() et groupBy().count() seront rapides
```

2. Transformations mapper sur les RDD

map() — Transformation élément par élément

1 élément → 1 élément · taille du RDD inchangée



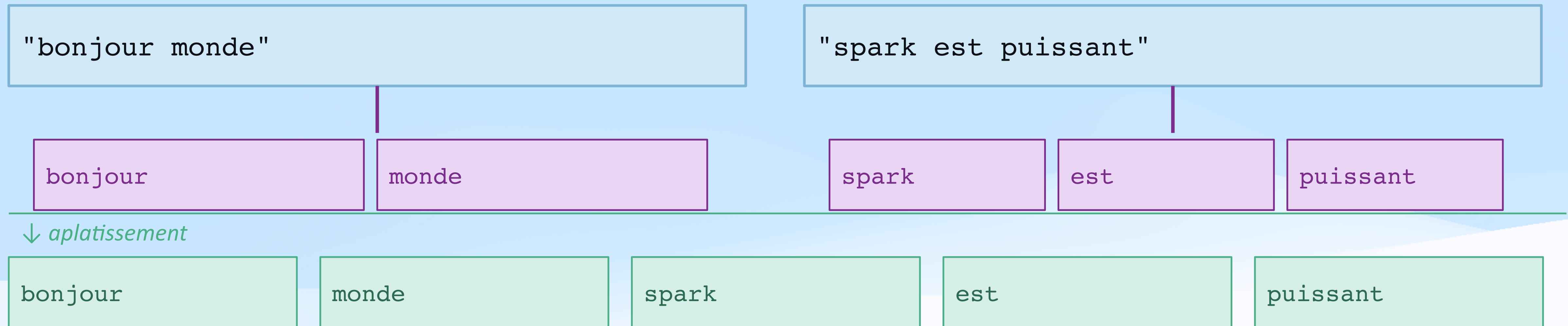
```
rdd = sc.parallelize([1,2,3,4,5])

# Lambda
rdd_double = rdd.map(lambda x: x * 2)
rdd_double.collect()    # → [2, 4, 6, 8, 10]

# Avec une fonction nommée (tuples clé-valeur)
rdd_t = sc.parallelize([("Alice", 1500.0), ("Bob", 800.0)])
rdd_t.map(lambda t: (t[0], t[1], t[1]*0.2)).collect()
# → [('Alice', 1500.0, 300.0), ('Bob', 800.0, 160.0)]
```

flatMap() — map + aplatissement

1 élément → N éléments · le résultat est aplati en un seul RDD



```
# map()      → RDD de listes  (3 éléments)
rdd.map(lambda l: l.split(' ')).collect()  # [['bonjour', 'monde'], ['spark', 'est', 'puissant'], ...]

# flatMap() → RDD de mots   (8 éléments)
rdd.flatMap(lambda l: l.split(' ')).collect()  # ['bonjour', 'monde', 'spark', 'est', 'puissant', ...]

# Filtrage + expansion combinés : retourner [] pour 'supprimer' un élément
rdd_nombres.flatMap(lambda x: [x**0.5] if x>=0 else [])
```


mapPartitions() et filter() sur RDD

mapPartitions() — transformation par partition entière

```
# map() : 1 connexion BDD par élément ❌  
rdd.map(lambda x: connexion_bd.query(x)) # connexion ouverte et fermée à chaque row  
  
# mapPartitions() : 1 connexion par partition ✅  
def traiter_partition(elements):  
    connexion = ouvrir_connexion()  
    resultats = [connexion.query(e) for e in elements]  
    connexion.fermer() ; return resultats  
rdd_resultat = rdd.mapPartitions(traiter_partition) # idéal pour charger un modèle ML
```

filter() — sélection conditionnelle

```
rdd = sc.parallelize(range(1, 21))  
rdd.filter(lambda x: x % 2 == 0).collect() # → [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]  
rdd.filter(lambda x: x > 10).collect() # → [11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
```

mapPartitionsWithIndex() : variante de mapPartitions() qui reçoit en plus le numéro de partition — utile pour le débogage ou les traitements différenciés par partition.

3. Transformations mapper sur les DataFrames

select()

Projection de colonnes (≡ SQL SELECT)

```
# Sélection simple
df.select("nom", "ville")

# Avec transformations inline
df.select(

    "nom", (F.col("montant")*1.2).alias("ttc"),
    F.upper(F.col(
    "ville")),
    F.length(F.col(
    "nom")).alias("len")
)
df.drop(
    "id").show()    # exclure une colonne
```


withColumn()

Ajout ou modification de colonne

```
# Colonne calculée
df.withColumn("ttc", F.col("montant") * 1.2)

# CASE WHEN
df.withColumn("cat",
  F.when(F.col(
    "montant") > 2000, "GOLD")
    .when(F.col(
    "montant") > 1000, "SILVER")
    .otherwise(
    "BRONZE"))

# Cast de type
df.withColumn("annee_str",
  F.col("annee").cast("string"))
```

withColumnRenamed() : `df.withColumnRenamed("ancien_nom", "nouveau_nom")` — pour renommer sans transformer.
En Spark 3.4+, préférer `df.withColumnsRenamed({...})` pour renommer plusieurs colonnes en un seul appel.

Concaténation

F.concat() et F.concat_ws(), identiques aux fonctions SQL

```
df.withColumn("nom_ville", F.concat(F.col("nom"), F.lit(" "), F.col("ville"), F.lit(""))) # →  
'Alice (Paris)'
```


filter() / where() — Filtrage de lignes

filter() et where() sont strictement synonymes en Spark

```
# Syntaxe F.col()
df.filter(F.col("montant") > 1000)

# Syntaxe SQL string
df.where("ville='Paris' AND montant>500")

# Conditions multiples
df.filter(
    (F.col("montant") > 500) &
    (F.col("ville").isin(["Paris", "Lyon"])) &
    F.col("nom").isNotNull()
)

# Négation
df.filter(~F.col("ville").isin(["Nantes"]))
```

```
# Filtres sur chaînes
df.filter(F.col("nom").startswith("A"))
df.filter(F.col("ville").contains("aris"))
df.filter(F.col("nom").rlike("^[A-E].*")) #
Regex

# Filtres sur dates
df.filter(F.col("date") >= F.lit("2024-01-01"))
df.filter(F.year(F.col("date")) == 2024)

# Valeurs nulles
df.filter(F.col("nom").isNotNull())
df.filter(F.col("note").between(0, 20))
```

Opérateurs logiques : & (ET) | | (OU) | ~ (NON) — toujours entre parenthèses car précedence Python différente de SQL.

Fonctions natives — pyspark.sql.functions

Toujours préférer F. aux UDF — optimisées par Catalyst, exécutées dans la JVM*

Mathématiques

- `F.abs(col)`
- `F.round(col, n)`
- `F.ceil(col)` / `F.floor(col)`
- `F.sqrt(col)` / `F.pow(col, n)`
- `F.log(col)` / `F.exp(col)`

Chaînes de caractères

- `F.upper(col)` / `F.lower(col)`
- `F.trim(col)` / `F.length(col)`
- `F.substring(col, pos, len)`
- `F.regexp_replace(col, p, r)`
- `F.concat(col1, col2, ...)`

Dates et temps

- `F.current_date()` / `F.current_timestamp()`
- `F.year(col)` / `F.month(col)`
- `F.datediff(col1, col2)`
- `F.date_add(col, n)`
- `F.to_date(col, 'yyyy-MM-dd')`

Structures & Nulls

- `F.size(col)` — taille Array ou Map
- `F.explode(col)` — 1 ligne → N lignes
- `F.array_contains(col, val)`
- `F.coalesce(col1, col2, ...)`
- `F.fillna(valeur, subset=[])`

Divers : `F.lit(valeur)` — constante · `F.col('nom')` — référence colonne · `F.when(...).otherwise(...)` — CASE WHEN · `F.monotonically_increasing_id()` — ID unique

4. Les UDF — User-Defined Functions

UDF — Création et utilisation

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType, DoubleType

# Méthode 1 : décorateur (recommandé)
@udf(returnType=StringType())
def normaliser_nom(nom):
    if nom is None: return None
    import re
    return ' '.join(w.capitalize() for w in re.sub(r'[^a-zA-Z ]', '', nom).split())

# Méthode 2 : enregistrement explicite
calculer_remise_udf = udf(calculer_remise, DoubleType())
```

⚡⚡⚡⚡ **Fonction native F.***

Exécutée dans la JVM — vectorisée et optimisable par Catalyst

⚡ **UDF Python classique**

Row → Python (Pickle) → exécution → JVM — Catalyst ne peut pas l'optimiser

Bonne pratique : ne jamais écrire une UDF qui duplique une fonction F.* existante. Mettre les imports en dehors de la fonction (pas à chaque appel).

5. Les Pandas UDF — UDF vectorisées

Pandas UDF — Traitement par batch via Apache Arrow

Transmettent des colonnes entières comme pandas.Series — bien plus efficace que les UDF classiques

UDF classique : [row1] → Python → JVM → [row2] → Python → JVM → ... sérialisation à chaque élément
Pandas UDF : [col_batch] → Python (pandas) → JVM sérialisation par batch via Apache Arrow

```
from pyspark.sql.functions import pandas_udf
import pandas as pd, numpy as np

# Pandas UDF SCALAR : Series → Series
@pandas_udf(DoubleType())
def normaliser_minmax(series: pd.Series) -> pd.Series:
    return (series - series.min()) / (series.max() - series.min())

df = df.withColumn("montant_norm", normaliser_minmax(F.col("montant")))

# Pandas UDF multi-colonnes
@pandas_udf(DoubleType())
def score_combine(montant: pd.Series, anciennete: pd.Series) -> pd.Series:
    return (montant * 0.7 + anciennete * 0.3).round(2)
```


Pandas UDF — Traitement par batch via Apache Arrow

Transmettent des colonnes entières comme pandas.Series — bien plus efficace que les UDF classiques

Type	Vitesse	Cas d'usage
Fonction native F.*	⚡⚡⚡⚡ (référence)	Toujours préférable si disponible
Pandas UDF (Arrow)	⚡⚡⚡ (~10-100x vs UDF)	Logique custom sur grandes colonnes
mapPartitions()	⚡⚡⚡	Chargement d'un modèle ML par partition
UDF Python classique	⚡	Logique simple, petit volume

6. Transformations sur les structures imbriquées

Colonnes Array

explode() et fonctions dédiées

```
# Colonne Array
data = [("Alice", ["Python", "Spark", "SQL"])]
df = spark.createDataFrame(data, ["nom", "skills"])

# Taille
df.withColumn("nb", F.size(F.col("skills")))

# Filtre
df.filter(F.array_contains(F.col("skills"), 'Spark'))

# explode : 1 ligne → N lignes
df.withColumn("skill", F.explode(F.col("skills"))) \
  .select(
    "nom", "skill").show()
```

Résultat typique d'un explode sur les compétences :

```
# Alice | Python
# Bob   | Java
# Alice | Spark
# Alice | SQL
# Claire | Python
# Claire | Spark
# Bob   | Scala    ← 1 ligne par (nom, compétence)
```

Colonnes Map

```
# Colonne Map (dictionnaire)
data = [("Alice", {"maths":18.0,"info":16.0})]

# Accéder à une clé
df.withColumn("note_maths",
  F.col(
    "notes")["maths"])

# Explorer le dictionnaire
df.withColumn("kv", F.explode(F.col("notes")))\
  .select(
    "nom","kv.key","kv.value")

# Clés / Valeurs
F.map_keys(F.col("notes")) # → ['maths','info']
F.map_values(F.col("notes")) # → [18.0, 16.0]
```

Cas d'usage typique de explode() : chaque compétence dans une colonne Array devient une ligne indépendante → permet un `groupBy( 'compétence' ).count( )` pour calculer la popularité de chaque compétence dans la base.

Résumé du module

Concept	Points clés à retenir
Lazy evaluation	Les transformations construisent un plan — rien n'est calculé avant une action
map()	1 élément → 1 élément, taille du RDD inchangée
flatMap()	1 élément → N éléments, aplatissement du résultat
mapPartitions()	Appliquée à la partition entière — idéal pour amortir des initialisations coûteuses
select()	Projection de colonnes avec transformations inline — alias, cast, calculs
withColumn()	Ajout ou modification d'une colonne — when().otherwise() pour les conditions
filter() / where()	Synonymes — filtrage par condition, regex, isin, isNull, between...
Fonctions natives F.*	Toujours préférer F.* aux UDF — optimisées par Catalyst, exécutées dans la JVM
UDF Python	Logique custom — coût de sérialisation Python ↔ JVM (10x plus lent que F.*)
Pandas UDF	UDF vectorisées via Apache Arrow — 10-100x plus rapides que les UDF classiques
explode()	Transforme une colonne Array/Map en plusieurs lignes — indispensable pour les structures imbriquées

Exercices

Ex. 1 — RDD et traitement de texte *(30 min)* Charger un roman (Projet Gutenberg) avec `textFile()`, nettoyer avec `flatMap()` (ponctuation, minuscules), compter les occurrences via `map()` + `reduceByKey()`, afficher les 20 mots les plus fréquents.

Ex. 2 — Pipeline DataFrame *(30 min)* Sur un CSV de clients : filtrer les emails valides (regex), normaliser les noms, ajouter `age_groupe` (18-25 / 26-40 / 41+) et `anciennete_jours` (`datediff`), sélectionner et renommer les colonnes utiles.

Ex. 3 — UDF vs fonction native *(20 min)* Écrire une UDF title case, l'équivalent avec `F.initcap()`, mesurer les temps d'exécution sur 100 000 lignes, comparer les plans d'exécution avec `.explain()`.

Ex. 4 — Pandas UDF et ML *(40 min)* Sur 10 000 lignes : Pandas UDF de standardisation z-score, Pandas UDF qui appelle un modèle scikit-learn pré-entraîné (`joblib.load()`), comparer les temps vs UDF classique.

Pour aller plus loin

Documentation

- API RDD PySpark : spark.apache.org/docs/latest/api/python/reference/api/pyspark.RDD.html
- Fonctions DataFrame : spark.apache.org/docs/latest/api/python/reference/pyspark.sql/functions.html
- Pandas UDF / Apache Arrow : spark.apache.org/docs/latest/api/python/user_guide/sql/arrow_pandas.html
- Apache Arrow (format colonnaire) : arrow.apache.org

Outil de test

spark-testing-base — framework de tests unitaires pour les transformations Spark — vérifie l'égalité de DataFrames, mock des SparkSessions, intégration pytest. github.com/holdenk/spark-testing-base

Rappel hiérarchie des performances

```
F.* (natif JVM) >> Pandas UDF (Arrow batch) >> mapPartitions() >> UDF classique (row-by-row)
```

Module suivant → Module 5 : Transformations de type reducer — groupBy, agg, join, window functions